HW4.3

RJ Cass

1

```
sleep <- sleep
sleep$incr <- sleep$extra > 0
d1 <- sleep[sleep$group == 1, ]
m1 <- nrow(d1)
d2 <- sleep[sleep$group == 2, ]
m2 <- nrow(d2)
```

(a) Create 95% confidence interval using the following 4 methods

(i) Use $\pi = 1/2$ to estimate SD

```
p1 <- 1/2
d1_var1 <- p1*(1-p1)/m1
d1_cint1 <- round(p1 + c(-1, 1)*qnorm(.975)*sqrt(d1_var1), 4)

d2_var1 <- p1*(1-p1)/m2
d2_cint1 <- round(p1 + c(-1, 1)*qnorm(.975)*sqrt(d2_var1), 4)
```

Drug 1 Interval 1: 0.1901, 0.8099

Drug 2 Interval 1: 0.1901, 0.8099

(ii) Use $\hat{\pi}$ to estimate mean and SD

```
d1_p2 <- mean(d1$incr)
d1_var2 <- d1_p2*(1-d1_p2)/m1
d1_cint2 <- round(d1_p2 + c(-1, 1)*qnorm(.975)*sqrt(d1_var2), 4)

d2_p2 <- mean(d2$incr)
d2_var2 <- d2_p2*(1-d2_p2)/m2
d2_cint2 <- round(d2_p2 + c(-1, 1)*qnorm(.975)*sqrt(d2_var2), 4)
```

Drug 1 Interval 2: 0.1901, 0.8099

Drug 2 Interval 2: 0.7141, 1.0859

(iii) Use $\pi = \frac{y+2}{m+4}$ to estimate SD (ie. $sd = \sqrt{\pi(1-\pi)/m}$)

```
d1_p3 <- (sum(d1$incr) + 2)/(m1 + 4)
d1_var3 <- d1_p2*(1-d1_p3)/m1
d1_cint3 <- round(d1_p3 + c(-1, 1)*qnorm(.975)*sqrt(d1_var3), 4)

d2_p3 <- (sum(d2$incr) + 2)/(m2 + 4)
d2_var3 <- d2_p3*(1-d2_p3)/m2
d2_cint3 <- round(d2_p3 + c(-1, 1)*qnorm(.975)*sqrt(d2_var3), 4)
```

Drug 1 Interval 3: 0.1901, 0.8099

Drug 2 Interval 3: 0.5314, 1.04

(iv) Using a logit link (ie. get CI on η, then transform that to a CI for π)

(I used AI a little bit here to help me fix some errors with my intervals)

```
d1_p4 <- mean(d1$incr)
d1_eta4 <- qlogis(d1_p4)
d1_se_eta4 <- 1 / sqrt(m1 * d1_p4 * (1 - d1_p4))
d1_cint_eta_4 <- d1_eta4 + c(-1, 1)*qnorm(.975)*d1_se_eta4
d1_cint4 <- round(plogis(d1_cint_eta_4), 4)

d2_p4 <- mean(d2$incr)
d2_eta4 <- qlogis(d2_p4)
d2_se_eta4 <- 1 / sqrt(m2 * d2_p4 * (1 - d2_p4))
d2_cint_eta_4 <- d2_eta4 + c(-1, 1)*qnorm(.975)*d2_se_eta4
d2_cint4 <- round(plogis(d2_cint_eta_4), 4)
```

Drug 1 Interval 4: 0.2245, 0.7755

Drug 2 Interval 4: 0.5328, 0.9861

(b) Comment on the differences/similarities between the confidence intervals. Why are they similar? Why are the different (and if different, which do I prefer)?

For Drug 1, intervals (i) and (ii) and (iii) are the same because in (ii) and (iii), the estimated probability happens to be 1/2 which is what we happened to use in (i). The value in (iv) is different because we are actually using a link function now which takes into account the binary nature of this response.

For Drug 2, all the intervals are different widths because we have different estimates of the mean, which when the mean is used to calculate the variance, means it impacts the width. (iv) is also different for the same reason as for Drug 1, due to the link function.

In both cases, I prefer the interval provided using the link function because it takes into account the binary nature of the response, and ensures our CI on Drug 2 does not go out of bounds (ie. > 1).

(c)

All of the confidence intervals imply that both drugs are likely to increase sleeping. However, none of the tests indicate that one of the drugs is likely to increase sleeping more than the other (all the confidence intervals overlap eachother between drugs 1 and 2).

2

$Y \sim Bin(m, \frac{e^\lambda}{1+e^\lambda})$. Show that m-Y is also Binomial and has induced parameter $\lambda' = -\lambda$

$f(y) = \sum_{i=1}^{m} \binom{m}{y_i} (\frac{e^\lambda}{1+e^\lambda})^{y_i} (1 - \frac{e^\lambda}{1+e^\lambda})^{m-y_i}$

$u = m - y, y = m - u, \frac{\partial}{\partial u} = |-1| = 1$

$$
\begin{aligned}
f(u) &= \sum_{i=1}^{m} \binom{m}{m-u_i} (\frac{e^\lambda}{1+e^{\lambda'}})^{m-u_i} (1 - \frac{e^\lambda}{1+e^\lambda})^{m-(m-u_i)} \\
&= \sum_{i=1}^{m} \binom{m}{m-u_i} (\frac{e^\lambda}{1+e^\lambda})^{m-u_i} (1 - \frac{e^\lambda}{1+e^\lambda})^{u_i} \\
&= \sum_{i=1}^{m} \binom{m}{m-u_i} (\frac{e^{\lambda'}}{1+e^{\lambda'}})^{u_i} (1 - \frac{e^{\lambda'}}{1+e^{\lambda'}})^{m-u_i}
\end{aligned}
$$

That last change to λ' comes from $1 - \frac{e^{\lambda'}}{1+e^{\lambda'}} = \frac{e^{\lambda}}{1+e^{\lambda}}$ if $\lambda' = -\lambda$, which confirms that part.

$\binom{m}{m-u_i} = \frac{m!}{(m-u_i)!(m-(m-u_i))!} = \frac{m!}{u_i!(m-u_i)!}$

Putting it all together we get: $\sum_{i=1}^{m} \frac{m!}{u_i!(m-u_i)!}(\frac{e^{\lambda'}}{1+e^{\lambda'}})^{u_i}(1 - \frac{e^{\lambda'}}{1+e^{\lambda'}})^{m-u_i}$ which is the Binomial PDF!

3

```
LI <- c(rep(8, 2), rep(10, 2), rep(12, 3), rep(14, 3), rep(16, 3), 18, rep(20, 3), rep(22, 2
Remissions <- c(rep(0, 2), rep(0, 2), rep(0, 3), rep(0, 3), rep(0, 3), 1, c(1, 1, 0), c(1,0)
dat <- data.frame(LI, Remissions)
```

(a)

```
set.seed(1337)
mod.logit <- glm(Remissions~LI, data=dat, family=binomial)
mod.probit <- glm(Remissions~LI, data=dat, family=binomial(link="probit"))
mod.cloglog <- glm(Remissions~LI, data=dat, family=binomial(link="cloglog"))
mod.loglog <- glm((1-Remissions)~LI, data=dat, family=binomial(link="cloglog"))

logit_aic <- summary(mod.logit)$aic
probit_aic <- summary(mod.probit)$aic
cloglog_aic <- summary(mod.cloglog)$aic
loglog_aic <- summary(mod.loglog)$aic

data.frame(link = c('logit', 'probit', 'cloglog', 'loglog'), aic = round(c(logit_aic, probit_
```

```
     link    aic
1   logit 30.073
2  probit 29.854
3 cloglog 30.995
4  loglog 28.499
```

```
library(boot)
library(knitr)
```

```r
#Mean Prediction Error
cost <- function(y, prob){mean(abs(y-prob) > 0.5)} #right/wrong prediction
cv.logit <- cv.glm(dat, mod.logit, cost=cost, K=5)$delta[1]
cv.probit <- cv.glm(dat, mod.probit, cost=cost, K=5)$delta[1]
cv.cloglog <- cv.glm(dat, mod.cloglog, cost=cost, K=5)$delta[1]
cv.loglog <- cv.glm(dat, mod.loglog, cost=cost, K=5)$delta[1]

#MSE loss
cv.logit.mse <- cv.glm(dat, mod.logit,  K=5)$delta[1]
cv.probit.mse <- cv.glm(dat, mod.probit, K=5)$delta[1]
cv.cloglog.mse <- cv.glm(dat, mod.cloglog, K=5)$delta[1]
cv.loglog.mse <- cv.glm(dat, mod.loglog, K=5)$delta[1]

#Sensitivity: proportion of y=1 correctly predicted (prob > 0.5)
sens.cost <- function(y, prob){mean(prob[y==1] > 0.5)}
cv.logit.sens <- cv.glm(dat, mod.logit, cost=sens.cost, K=5)$delta[1]
cv.probit.sens <- cv.glm(dat, mod.probit, cost=sens.cost, K=5)$delta[1]
cv.cloglog.sens <- cv.glm(dat, mod.cloglog, cost=sens.cost, K=5)$delta[1]

#Specificity: proportion of y=0 correctly predicted (prob <= 0.5)
spec.cost <- function(y, prob){mean(prob[y==0] <= 0.5)}
cv.logit.spec <- cv.glm(dat, mod.logit, cost=spec.cost, K=5)$delta[1]
cv.probit.spec <- cv.glm(dat, mod.probit, cost=spec.cost, K=5)$delta[1]
cv.cloglog.spec <- cv.glm(dat, mod.cloglog, cost=spec.cost, K=5)$delta[1]

#For the loglog model the fitted probability is for 1-y, so flip the thresholds
sens.cost.loglog <- function(y, prob){mean(prob[y==0] < 0.5)}
spec.cost.loglog <- function(y, prob){mean(prob[y==1] > 0.5)}
cv.loglog.sens <- cv.glm(dat, mod.loglog, cost=sens.cost.loglog, K=5)$delta[1]
cv.loglog.spec <- cv.glm(dat, mod.loglog, cost=spec.cost.loglog, K=5)$delta[1]

mycv <- data.frame(link=c("logit", "probit", "cloglog", "loglog"), abs.cost=c(cv.logit,cv.pr
kable(mycv)
```

link	abs.cost	mse.cost	sensitivity	specificity
logit	0.2592593	0.1968694	0.3148148	0.8148148
probit	0.2592593	0.1798883	0.2962963	0.8814815
cloglog	0.2962963	0.1945725	NaN	0.8814815
loglog	0.2222222	0.2003007	0.5555556	0.8981481

Looking at AIC we see that log-log has the lowest AIC. We also see that it has the highest

sensitivity and specificity. For these reasons I am picking to use the log-log link (also because Dr. Berret said to, but coincidentally my measures said to as well).

(b)

```
summary(mod.loglog)
```

```
Call:
glm(formula = (1 - Remissions) ~ LI, family = binomial(link = "cloglog"),
    data = dat)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  2.42322    0.82711   2.930  0.00339 **
LI          -0.11399    0.04246  -2.685  0.00726 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 34.372  on 26  degrees of freedom
Residual deviance: 24.499  on 25  degrees of freedom
AIC: 28.499

Number of Fisher Scoring iterations: 8
```

(I used AI to know how the heck to interpret this thing)

We get an estimate of ~ -.114. This means that if LI increases by 1%, the expected probability of NOT going into remission (because we are fitting 1-y) is expected to decrease by ~ .114%. This means that for every 1% increase in LI, we would expect a .114% INCREASE in the probability of remission.

According to the model summary we have a p-value of ~.007 so yes, we would conclude that LI does appear to be statistically signifcant.

(c)

Log-Log link function: $ln(-ln(1-p)) = \beta_0 + \beta_1 * LI$

$\implies LI = \frac{ln(-ln(1-p))-\beta_0}{\beta_1}$

Because we are just looking for $p = .5$ we don't really need to worry about the whole 1-y stuff used in the model (ie. don't need to do the negative of $beta_1$ or anything)

```
beta <- coef(mod.loglog)

LI_target <- round((log(-log(1 - 0.5)) - beta[1]) / beta[2], 4)
```

Target LI to get $\hat{\pi} = .5$: 24.4738

(d)

(I used AI here to help me figure out how to get the prediction/transform it)

```
pred <- predict(mod.loglog, newdata = data.frame(LI = 8), se.fit = TRUE)

# Calculate 95% CI bounds on the link scale
link_cint <- pred$fit  + c(-1, 1)* qnorm(0.975) * pred$se.fit

# Back-transform directly to the chance of REMISSION
# Link upper maps to remission lower due to negative exponential changing the directionality
remission_cint <- c(exp(-exp(link_cint[2])), exp(-exp(link_cint[1])))
```

LI = 8 chance of remission CI: 3.3365505×10^{-6}, 0.1960862

So, we are 95% confident that the probability of a patient entering remission at an LI of 8 lies between .000003% and .196%.